BIG DATA — COMPLETE REVISION SHEET

C-CAT Section B | Phase 1: Topics 1–10 | All Key Concepts, Comparisons & MCQ Facts

Designed for 1-hour exam revision — read top-to-bottom, topic by topic

TOPIC 1 · BIG DATA CONCEPTS — THE 5 VS

What Big Data is and why it needs special tools

◆ What is Big Data?

Big Data refers to datasets that are so enormous in size, so fast in arrival, or so varied in type that traditional tools — like Microsoft Excel or a standard relational database — simply cannot store, process, or analyse them. The world generates massive amounts of data every second: social media posts, financial transactions, sensor readings, videos, and more. To handle this, a completely different set of tools and techniques was needed. Big Data is defined by five key characteristics called the 5 Vs, each describing a different challenge dimension.

◆ The 5 Vs — Definition and Context

Volume is about the sheer SIZE of data. We are not talking megabytes or gigabytes — Big Data operates at terabytes, petabytes, and exabytes. For example, Facebook stores over 100 petabytes of photos and videos, and Google processes roughly 20 petabytes of data every single day. Volume is why normal databases fail — they run out of storage and computing power.

Velocity is about the SPEED at which data is generated and must be processed. Data today does not just sit still — it arrives in a continuous, high-speed stream. Every minute, 6 million Google searches happen, over 500,000 tweets are posted, and 500 hours of video are uploaded to YouTube. Systems must either process this data instantly as it arrives (streaming) or collect and process it in scheduled chunks (batch). You will study both approaches in Topic 3.

Variety is about the different TYPES or FORMATS of data. Traditional databases only handle neat rows and columns. But real-world data comes as photos, videos, voice notes, emails, GPS coordinates, and JSON files — all very different formats. Variety is why over 80% of today's data cannot fit into a traditional database without special handling. You will study the three data types in depth in Topic 2.

Veracity is about the TRUSTWORTHINESS of data. When data arrives from millions of sources at high speed, much of it is noisy, incomplete, or inaccurate — fake reviews, spam, typos, inconsistent formats across systems. Veracity asks: how reliable is this data? Big Data systems must handle messy, uncertain data rather than assuming everything is clean.

Value is the most important V — the USEFULNESS you extract from data. Data by itself is worthless. It only matters if you can turn it into insights that drive decisions. A hospital collecting patient data has no benefit until someone analyses it and discovers, for example, that combining two specific drugs improves recovery rates by 30%. Value is the entire reason Big Data exists — all the cost of storing and processing data is justified only if useful insights come out.

V	One Word	Key Question	Exam Example
Volume	Size	How much data?	Facebook 100 PB photos; Google 20 PB/day
Velocity	Speed	How fast does data arrive?	6M Google searches/min; 500K tweets/min
Variety	Type	What formats exist?	Structured, Semi-structured, Unstructured
Veracity	Trust	How accurate/reliable?	Fake reviews, spam, typos, multi-language noise
Value	Usefulness	What insight can we get?	Hospital finding better drug combination

■ **Volume = SIZE of data (petabytes). Value = USEFULNESS extracted. These are commonly swapped in exams.**

■ The 3 Vs (Volume, Velocity, Variety) were defined by Doug Laney in 2001. Veracity and Value were added later by IBM. A 6th V — Variability — sometimes appears: it means the meaning of data keeps changing over time.

TOPIC 2 · TYPES OF DATA

Structured, Semi-Structured & Unstructured

◆ Why Data Types Matter

One of the 5 Vs — Variety — tells us that Big Data comes in many different forms. Not all data is the same. Some data is neatly organised in rows and columns, some has partial labels but no rigid structure, and some has no structure whatsoever. Understanding these three types is critical

because each type requires completely different storage systems and processing tools. Traditional databases can only handle one type. Big Data systems must handle all three.

◆ Structured Data

Structured data is organised into a fixed format — specifically rows and columns, like a spreadsheet or a database table. Every piece of data fits into a predefined field. For example, a bank's transaction database has columns like Account No., Name, Balance, and Date — and every record follows exactly that structure. Structured data is the easiest to store, search, and analyse. It is stored in RDBMS (Relational Database Management Systems) like MySQL or Oracle, which you will study in Topic 5. The key limitation: structured data represents only about 20% of all data generated in the world.

◆ Semi-Structured Data

Semi-structured data does not fit into a rigid table but still has some organisational markers — like tags, labels, or key-value pairs. A key-value pair simply means a label paired with its content, for example: 'Name': 'Rahul'. JSON (JavaScript Object Notation) and XML (eXtensible Markup Language) are the two most important semi-structured formats for the exam. Emails are also semi-structured — the To, From, Subject fields are structured, but the email body is free-form text. Semi-structured data acts as a bridge between the other two types — it gives flexibility while retaining some organisation. It is stored in NoSQL databases (Topic 4).

◆ Unstructured Data

Unstructured data has no predefined format whatsoever. Photos, videos, audio recordings, social media posts, WhatsApp messages, and PDF documents are all unstructured. A computer cannot directly read meaning from a photo without special AI techniques like Computer Vision. It cannot understand speech without Natural Language Processing (NLP). This is the biggest challenge in Big Data — and the biggest opportunity. Unstructured data makes up approximately 80–90% of all data generated in the world today. It is stored in Data Lakes, which you will encounter in Topic 6.

Feature	Structured	Semi-Structured	Unstructured
Format	Fixed rows & columns	Tags / key-value pairs	No fixed format
Examples	SQL tables, Excel	JSON, XML, Emails, HTML	Images, Videos, Audio, Social posts
Storage System	RDBMS (MySQL, Oracle)	NoSQL DBs, files	Data Lakes, special storage
Easy to Search?	✓ Very easy	Somewhat easy	✗ Hard — needs AI
% of World Data	~20%	Small portion	~80–90%
Processing Tools	SQL, Excel	JSON/XML parsers	AI, NLP, Computer Vision

■ **Excel file = STRUCTURED (rows & columns). Email = SEMI-STRUCTURED (fixed headers + free body). Do not confuse these.**

■ Metadata = data about data. A photo has metadata: date taken, GPS location, camera model. Metadata helps organise unstructured files without reading the file content itself.

TOPIC 3 · BIG DATA PROCESSING — BATCH & STREAMING

How raw data is converted into useful results

◆ What is Data Processing?

Data processing means taking raw data and converting it into something useful — a report, a result, a decision, or an alert. Raw data by itself is just numbers and text. Processing gives it meaning. In Big Data, where data arrives at massive scale and speed, how and when you process it becomes a critical design decision. There are two fundamentally different approaches: Batch Processing and Streaming Processing. Each has different strengths, different tools, and different use cases.

◆ Batch Processing — Collect First, Process Later

In Batch Processing, data is not processed immediately. Instead, it is collected and stored over a period of time — could be an hour, a day, or a month. Then, at a scheduled time, the entire collected dataset is processed together in one large job. Think of a bank that collects all your transactions throughout the month and then calculates your monthly statement at month-end. The calculation happens once, on all the data together. This is efficient for large volumes but introduces a time delay — called latency — between when data arrives and when results are ready. The primary tool for Batch Processing is Apache Hadoop (Topic 9).

◆ Streaming Processing — Process Instantly as Data Arrives

In Streaming Processing, data is processed the moment it arrives — there is no waiting, no collecting first. Each data point is handled instantly, within milliseconds to seconds. Think of a UPI fraud detection system: when you make a payment, the bank's system checks for fraud right now, before approving the transaction. If it waited until midnight to check, the money would already be gone. Streaming is essential for any use case where delay causes harm or loss. The primary tools are Apache Kafka (delivers the stream) and Apache Flink or Spark Streaming (processes the stream). Latency is extremely low in streaming systems.

Feature	Batch Processing	Streaming Processing
When processed?	After collecting over time	Instantly as each event arrives
Latency	HIGH — hours or days of delay	LOW — milliseconds to seconds
Data Volume	Large historical datasets	Continuous live data
Cost & Complexity	Lower cost, simpler to build	Higher cost, more complex
Best For	Monthly reports, billing, payroll	Fraud detection, live tracking, stock market
Main Tool	Apache Hadoop (MapReduce)	Apache Kafka, Flink, Spark Streaming
Real Example	Bank monthly statement	UPI fraud detection in real time

■ **Apache Spark does BOTH batch AND streaming via micro-batching — one of the most commonly tested facts.**

■ Micro-batch = Spark collects data every few seconds, processes that tiny batch, then repeats. It is near-real-time, not true real-time. Apache Flink = true real-time (processes each event the instant it arrives).

■ Lambda Architecture = system that combines both: a Batch Layer for historical accuracy + a Speed Layer for real-time + a Serving Layer that merges both results for users.

■ Latency = the time gap between data being generated and results being available. Batch has HIGH latency. Streaming has LOW latency.

TOPIC 4 · NOSQL, BASE PROPERTIES & CAP THEOREM

Flexible databases built for Big Data

◆ Why NoSQL Was Needed

Traditional SQL databases (RDBMS) were built for structured data with fixed schemas. They struggle with two Big Data realities: they cannot store unstructured or semi-structured data, and they scale vertically (upgrading one machine has limits). NoSQL — which stands for 'Not Only SQL' — was built to solve both problems. NoSQL databases are flexible (no fixed schema required), can store any data type, and scale horizontally (just add more machines). They sacrifice some of the strict guarantees of traditional databases in exchange for speed, scale, and flexibility.

◆ 4 Types of NoSQL Databases

Type	How Data is Stored	Examples	Best Use Case
Key-Value Store	Unique key paired with a value (like a dictionary)	Redis, DynamoDB	Shopping carts, user sessions, caching
Document Store	JSON/XML documents — each can have different fields	MongoDB, CouchDB	User profiles, e-commerce product catalogues
Column-Family Store	Data grouped by columns, not rows	Cassandra, HBase	Time-series data, large-scale analytics
Graph Database	Nodes (entities) connected by Edges (relationships)	Neo4j, Neptune	Social networks, fraud detection networks

◆ BASE Properties — How NoSQL Databases Behave

Traditional SQL databases follow ACID rules (strict consistency, covered in Topic 5). NoSQL databases follow a more relaxed set of rules called BASE, which prioritises availability and speed over perfect instant accuracy. Understanding BASE is critical because it explains why NoSQL databases can be fast and always available — they make a deliberate trade-off on consistency.

- **B — Basically Available:** The system always responds to requests, even if the data returned is not perfectly up to date. The website stays up and answers, even during issues.
- **A — Soft State:** The current state of data may change over time even without new user input — because the system is still syncing and updating copies across multiple machines.
- **E — Eventually Consistent:** All copies of the data across all machines will eventually match — but not necessarily at the exact same instant. Example: when you post a tweet, it may not appear for all users worldwide at the same millisecond, but within a few seconds everyone sees it.

◆ CAP Theorem — Eric Brewer, 2000

The CAP Theorem states that any distributed database system — one spread across multiple machines — can only fully guarantee TWO out of these three properties at the same time. This is not a design flaw; it is a mathematical impossibility to have all three simultaneously in a distributed system. Understanding CAP helps you understand why different databases make different trade-offs.

- **C — Consistency:** Every read returns the most recent write. All nodes (machines) show identical data at the same moment.
- **A — Availability:** The system always responds to every request, even if some machines are down.
- **P — Partition Tolerance:** The system keeps functioning even if the network connection between some machines breaks (a 'partition').

In practice, network failures WILL happen — so Partition Tolerance (P) is always required. The real choice is between CP (sacrifice Availability) and AP (sacrifice Consistency).

Database	CAP Choice	Practical Meaning
MongoDB	CP	Stays consistent during failures — may stop responding temporarily
Cassandra	AP	Always responds — may return slightly outdated data
HBase	CP	Prioritises consistency over availability
CouchDB	AP	Always available — eventual consistency
Redis	CP	Prioritises consistency

■ **CAP = C (Consistency), A (Availability), P (Partition Tolerance). The A stands for Availability — NOT 'I'. This is a common trap.**

■ **BASE data is NOT wrong or unreliable — it is temporarily out of sync but always moving toward correctness (Eventually Consistent).**

■ NoSQL = 'Not Only SQL' — not 'No SQL at all.' Schema-less means you can add new fields at any time without redesigning the whole database. Horizontal scaling = add more machines (NoSQL). Vertical scaling = upgrade one machine (SQL).

TOPIC 5 · RDBMS, ACID PROPERTIES & BASIC SQL

Traditional relational databases and their guarantees

◆ What is RDBMS?

RDBMS stands for Relational Database Management System. It stores data in tables (rows and columns) and allows multiple tables to be related to each other through common fields. For example, a Student table and a Course table can be linked through a Course ID — every student record points to a course record without copying all the course details. SQL (Structured Query Language) is the language used to interact with RDBMS — to store, retrieve, update, or delete data. RDBMS is the backbone of day-to-day operational systems like banking, hospital records, and e-commerce order management.

◆ Key RDBMS Terms

Term	Simple Meaning	Example
Table / Relation	Grid of rows and columns for one entity	Student table, Course table
Row / Record / Tuple	One single entry in a table	Details of one student
Column / Attribute	One property being tracked	Name, Age, Roll Number
Primary Key (PK)	Unique identifier — no NULL, no duplicate allowed	Roll No in Student table
Foreign Key (FK)	Column linking to PK of another table (the relationship)	Course_ID in Student table
Schema	Blueprint of table — columns + data types + rules, defined before data entry	Defined before inserting any data
NULL	Absence of value — not zero, not empty, but unknown	Age field left blank

◆ ACID Properties — What Makes RDBMS Reliable

A transaction is a sequence of database operations treated as a single unit — for example, transferring money involves two steps: deduct from one account AND add to another. ACID properties guarantee that transactions are processed reliably even if something goes wrong mid-way (power cut, system crash, two users acting simultaneously). Without ACID, data corruption would be constant.

Property	One Word	What It Guarantees	If Violated — What Happens
Atomicity	All-or-Nothing	Every step completes OR entire transaction is rolled back (undone)	Half-transfer: money deducted but not credited — vanishes
Consistency	Valid State	Transaction moves DB from one valid state to another. Rules never broken	Balance goes below zero — invalid data in DB
Isolation	Independence	Concurrent transactions do not interfere with each other	Two users withdraw same money simultaneously — double spend
Durability	Permanent	Committed (completed) data survives crashes permanently	Confirmed transaction disappears after power cut

◆ SQL Command Categories

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define and modify table structure
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Work with data inside tables
DCL	Data Control Language	GRANT, REVOKE	Control user access permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage and control transactions

■ **DROP = permanently deletes the table AND its structure. TRUNCATE = deletes all rows but keeps the table structure intact.**

■ **DELETE with WHERE removes specific rows. TRUNCATE removes ALL rows with no conditions — know the difference.**

■ COMMIT = permanently saves a completed transaction. ROLLBACK = undoes a failed or cancelled transaction. These are TCL commands.

■ Normalisation = organising a database to reduce data redundancy (unnecessary repetition). Split large tables into smaller related ones. Normal Forms: 1NF, 2NF, 3NF.

TOPIC 6 · DATA WAREHOUSE, OLAP & OLTP

From daily transactions to business intelligence

◆ The Big Picture — Three Interconnected Systems

To understand Data Warehouses, you need to understand the flow of data in a real business. Every day, thousands of transactions happen — sales, payments, bookings, registrations. These are recorded by OLTP systems (Online Transaction Processing) — fast databases built for recording data quickly. But business leaders need to ask bigger questions: which product sold most over 5 years? Which region grew fastest? A live transaction database cannot answer these efficiently. So data is collected from OLTP systems, cleaned, and stored in a Data Warehouse — a separate, historical database optimised for analysis. OLAP (Online Analytical Processing) is the system that runs complex analytical queries on this Data Warehouse.

◆ OLTP — Online Transaction Processing

OLTP systems handle the operational, day-to-day transactions of a business. They are optimised for writing data quickly — inserting a new order, updating a balance, deleting a cancelled booking. Each transaction involves only a few rows (1–10 typically), must complete in milliseconds, and the data must always be current and accurate. OLTP uses RDBMS (MySQL, Oracle) with normalised design (data split into many related tables to avoid redundancy). Examples: ATM transactions, IRCTC ticket booking, Amazon checkout, UPI payments.

◆ Data Warehouse — Centralised Historical Storage

A Data Warehouse is a large, centralised storage system that collects data from multiple OLTP sources, cleans it, and stores it specifically for analysis — not for day-to-day transactions. The key word is historical: a Data Warehouse keeps months or years of data so analysts can study trends over time. Data enters the warehouse through an ETL process (Extract, Transform, Load — covered in Topic 7). Unlike OLTP, a Data Warehouse uses denormalised design — data is intentionally combined into wider tables for faster reading, even if some data is repeated. Popular tools: Amazon Redshift, Google BigQuery, Snowflake, Teradata.

◆ OLAP — Online Analytical Processing

OLAP is the system that runs complex analytical queries on the Data Warehouse. These queries involve multiple dimensions (category perspectives like Time, Location, Product) and millions of rows. For example: 'Show total smartphone sales by state for each quarter of the last 3 years.' OLAP data is conceptually imagined as a cube where each axis represents one dimension. You can manipulate this cube using five operations, all of which are directly exam-worthy.

Feature	OLTP	OLAP
Full Form	Online Transaction Processing	Online Analytical Processing
Purpose	Day-to-day operations	Analysis and decision-making
Operations	INSERT, UPDATE, DELETE	SELECT (complex multi-table queries)
Rows per Query	Few rows (1–10)	Millions of rows
Response Time	Milliseconds	Seconds to minutes
Data Type	Current, real-time data	Historical data (months/years)
DB Design	Normalised (avoid redundancy)	Denormalised (combined for speed)
Users	Clerks, cashiers, customers	Analysts, managers, executives
Backed By	RDBMS — MySQL, Oracle	Data Warehouse — Redshift, BigQuery
Real Examples	ATM, IRCTC booking, UPI	Sales trend reports, 5-year forecasting

◆ OLAP Operations — All 5 Are Exam-Worthy

OLAP data is visualised as a multi-dimensional cube. Each axis of the cube is a dimension (Time, Location, Product, etc.). These five operations let you navigate and analyse the cube from different angles.

Operation	Direction	What It Does	Example
Roll Up	Bottom → Top	Summarise from detail to higher level	Monthly sales → Quarterly → Yearly
Drill Down	Top → Bottom	Expand from summary down to detail	Yearly → Quarterly → Monthly → Daily

Operation	Direction	What It Does	Example
Slice	One dimension	Fix one dimension's value — gives a 2D flat view	Look at only Year 2023 data
Dice	Multi-dim	Select specific values across multiple dimensions	Smartphones in Maharashtra in Q1 2023
Pivot	Rotate	Swap rows and columns to view from different angle	Switch time axis with region axis in report

◆ Schema Types Inside a Data Warehouse

Data inside a Data Warehouse is organised using schemas. The most common is Star Schema. A Fact Table sits in the centre holding measurable data (sales amounts, quantities, revenue). Dimension Tables surround it holding descriptive context (product names, customer details, dates, locations). This looks like a star. Snowflake Schema is a variation where Dimension Tables are further split into sub-tables (normalised), saving storage but requiring more JOINs and slowing queries.

Schema	Structure	Query Speed	Storage	Use When
Star Schema	1 Fact Table + flat Dimension Tables	Faster — fewer JOINs	More (some redundancy)	Query speed is priority
Snowflake Schema	Fact Table + normalised (split) Dimension Tables	Slower — more JOINs	Less (no redundancy)	Storage saving is priority
Galaxy Schema	Multiple Fact Tables sharing Dimensions	Complex	Enterprise scale	Large enterprise DW

■ **OLTP = Normalised design. OLAP / Data Warehouse = Denormalised design. This is frequently tested — do not swap them.**

■ **Roll Up ≠ Drill Down. Roll Up = going UP to a bigger summary. Drill Down = going DOWN into more detail.**

■ OLAP Types: ROLAP (runs on relational DB, uses SQL), MOLAP (stores in special cube format, fastest), HOLAP (hybrid of both).

■ Data Mart = smaller, department-focused version of a Data Warehouse (e.g. Sales Mart, HR Mart).

■ Fact Table = measurable data (amounts, quantities). Dimension Table = descriptive context (names, dates, locations).

TOPIC 7 · ETL VS ELT

How data moves from source systems to storage

◆ What is ETL and Why Does It Exist?

ETL stands for Extract, Transform, Load. It is the process that feeds data into a Data Warehouse. Raw data from OLTP systems is messy — different formats, different databases, missing values, duplicates. You cannot just dump this raw data into a Data Warehouse. ETL solves this: first Extract data from multiple sources, then Transform it (clean, standardise, merge) in a temporary area called the Staging Area, then Load the clean data into the Data Warehouse. ETL ensures only clean, consistent, structured data enters the warehouse.

◆ What is ELT and Why Was It Created?

ELT stands for Extract, Load, Transform — same letters, different order. In ELT, raw data is loaded directly into the target system first (usually a Data Lake), and transformation happens later inside that system on demand. ELT was created for the modern cloud era, where storage is cheap and cloud Data Warehouses (like BigQuery and Snowflake) are powerful enough to run transformations themselves. ELT is also better for Big Data because it can handle unstructured and semi-structured data — ETL struggles with anything that isn't structured. The key advantage of ELT: raw data is always preserved, so you can re-process it differently in the future.

Feature	ETL	ELT
Full Form	Extract → Transform → Load	Extract → Load → Transform
When Transform?	BEFORE loading — in Staging Area	AFTER loading — inside target system
Target System	Data Warehouse	Data Lake / Cloud Data Warehouse
Data Types	Mostly structured data only	Structured + Semi-structured + Unstructured
Raw Data Preserved?	NO — discarded after transformation	YES — always available for reprocessing
Loading Speed	Slower — must transform before loading	Faster — load raw data immediately
Best For	Traditional, structured, defined reporting	Big Data, unstructured, flexible AI analysis
Era	Traditional / On-premise systems	Modern / Cloud-based systems
Example Tools	Informatica, Talend, IBM DataStage	Apache Spark, dbt, Google Dataflow

◆ ETL Steps in Detail

• **EXTRACT:** Pull data from multiple sources — OLTP databases (MySQL, Oracle), CSV files, web APIs, legacy mainframes. Each source may use a different format and database type.

• **TRANSFORM (in Staging Area):** The staging area is a temporary workspace between the source and the Data Warehouse. Here: clean errors and nulls, remove duplicates, standardise formats (dates, currencies, names), aggregate values, merge data from different sources, apply business rules (e.g. flag transactions above ■1 lakh as 'high value').

• **LOAD:** Two types — Full Load (entire dataset, used when setting up for the first time) or Incremental Load (only new/changed records since last run — used for regular updates like nightly refreshes).

■ **ELT is NOT just ETL with steps reordered for convenience. It is a fundamentally different architecture suited for cloud and Big Data environments.**

■ **ETL requires structured data. ELT handles ALL data types. Do not swap this in exam questions.**

■ GIGO = Garbage In, Garbage Out — if dirty data enters your Data Warehouse, analysis results will be wrong no matter how powerful your tools are.

■ Staging Area (ETL only) = temporary workspace between source and destination. Deleted/archived after data is loaded.

■ CDC = Change Data Capture — technique used in Incremental Load that tracks only changed rows since last ETL run, making the process efficient.

TOPIC 8 · DATA CLEANSING & DATA MODELING

Fixing dirty data and designing database blueprints

◆ What is Data Cleansing?

Data Cleansing (also called Data Cleaning or Data Scrubbing) is the process of detecting and correcting — or removing — inaccurate, incomplete, duplicate, or irrelevant records from a dataset. Real-world data collected from multiple sources is almost always dirty. Without cleansing, the principle of GIGO (Garbage In, Garbage Out) applies — no matter how powerful your analytics tools are, bad input data produces wrong output. Data Cleansing is the critical quality gate, typically done during the Transform step of ETL.

◆ Common Data Problems and How to Fix Them

Problem	What It Means	How to Fix It
Missing Values	Fields with no data (NULL or blank)	Delete row / Imputation (fill with mean, median, mode) / Flag as Unknown
Duplicate Records	Same real-world entity entered multiple times	Keep one record, delete duplicates (merge unique info first if needed)
Inconsistent Format	Same data stored in different formats across sources	Standardise to one format: dates → DD-MM-YYYY, names → Title Case
Invalid Data	Impossible values (age=450, rating=7.5 out of 5)	Correct if possible, delete, or flag for review
Outliers	Values abnormally far from all other values	Remove if data entry error; keep if genuine extreme case
Inconsistent Values	Same concept stored differently (M / Male / male / MALE)	Standardise to one canonical value
Misspellings	Typos in text fields: 'Mumbay', 'Mumabi'	Match against standard reference list and correct
Irrelevant Data	Data that is not needed for the analysis at hand	Filter out / remove from the working dataset

• **Imputation** = filling a missing value with the mean (average), median (middle value), or mode (most frequent value) of that column — rather than deleting the entire row.

• **Data Profiling** = Step 1 of cleansing — examine the dataset first to discover what problems exist before fixing them.

• **Data Validation** = final check after all cleaning — confirm data now meets quality standards.

• **Data Wrangling / Munging** = broader term: includes cleansing + reshaping + enriching data to make it usable.

◆ What is Data Modeling?

Data Modeling is the process of creating a blueprint — visual or logical — of how data should be organised, structured, and related within a database before actually building it. A Data Model defines what entities (real-world things) exist, what attributes (properties) each has, and how they relate to each other. Think of it as the architect's drawing before construction begins. You would never build a database without a model — just as you would never build a building without a blueprint.

◆ Three Levels of Data Modeling

Level	Name	Detail	What It Shows	Who Uses It
1	Conceptual	Abstract / Big picture	Entities + relationships only. No attributes, no technical detail.	Business managers, stakeholders

Level	Name	Detail	What It Shows	Who Uses It
2	Logical	Medium detail	Attributes + relationships defined. Technology-independent (no specific DB).	Data architects, analysts
3	Physical	Maximum detail	Exact columns, data types, constraints, indexes for a specific DB system.	Developers, DBAs

■ **Data Cleansing ≠ Data Transformation. Cleansing = fix quality problems (errors, duplicates). Transformation = restructure and reformat data for a specific analytical purpose.**

■ ERD = Entity Relationship Diagram — the visual diagram that represents a Data Model. Entities are rectangles, relationships are lines or diamonds. Used at Conceptual and Logical levels.

■ Entity = real-world object being tracked (Student, Course, Product). Attribute = a property of that entity (Name, Age, Price). Relationship = how two entities connect (Student enrolls in Course).

TOPIC 9 · BIG DATA FRAMEWORKS — HADOOP, HIVE & SPARK

The core tools that power Big Data at scale

◆ Apache Hadoop — The Foundation

Apache Hadoop was created by Doug Cutting and Mike Cafarella in 2006, inspired by research papers published by Google. Hadoop solved a fundamental problem: data was growing faster than any single computer could handle. Instead of buying one impossibly expensive super-computer, Hadoop's insight was to use hundreds or thousands of cheap, ordinary machines working together as one system — called a cluster. Hadoop has two core components: HDFS for storage and MapReduce for processing.

◆ HDFS — Hadoop Distributed File System

HDFS is how Hadoop stores data across the cluster. When a large file is stored, HDFS automatically splits it into fixed-size blocks (default: 128 MB each) and distributes these blocks across different machines (DataNodes). To prevent data loss if a machine fails, each block is copied to 3 machines by default (replication factor = 3) — this is called fault tolerance. A special master machine called the NameNode keeps track of where every block of every file lives, but does not store the actual data itself.

HDFS Component	Role	Key Facts
NameNode	Master — stores metadata only (the map of where blocks live)	Does NOT store actual data. Single master node.
DataNode	Worker — stores the actual data blocks	Many DataNodes across the cluster.
Block	Fixed-size chunk of a file	Default = 128 MB. Older versions = 64 MB.
Replication	Same block copied to multiple machines	Default replication factor = 3 (fault tolerance).

◆ MapReduce — Hadoop's Processing Engine

MapReduce is how Hadoop processes data — it breaks a large task into two phases and runs them in parallel across all machines. In the MAP phase, each machine processes its local data blocks and produces key-value pairs (labels paired with values). In the REDUCE phase, all pairs with the same key are grouped together and aggregated into final results. The critical limitation of MapReduce: after every Map and Reduce phase, results are written back to disk. The next phase reads from disk again. This constant reading and writing to disk makes MapReduce very slow — especially for multi-step tasks. This is exactly the problem Spark was built to solve.

◆ Apache Hive — SQL for Big Data

Hive was created by Facebook in 2008 and sits on top of Hadoop. Most business analysts know SQL but not Java (the language needed to write MapReduce jobs). Hive solves this: you write queries in HiveQL (a language nearly identical to SQL), and Hive automatically converts them into MapReduce jobs that run on Hadoop. You get the power of Hadoop without learning Java. Important: Hive uses Schema on Read — it does not enforce structure when data is stored, only when it is read. Hive is for batch analysis only, not real-time queries. Hive is NOT a database — it is a query layer on top of HDFS.

◆ Apache Spark — Fast In-Memory Processing

Spark was created at UC Berkeley's AMPLab in 2009. Its key innovation: instead of reading and writing to disk between every processing step (like MapReduce), Spark keeps intermediate data in RAM (the computer's fast temporary memory). RAM access is approximately 100 times faster than disk access. This makes Spark up to 100x faster than MapReduce for many tasks. Spark can handle both batch processing and streaming (via micro-batching), making it the most versatile Big Data processing engine available. Importantly, Spark does NOT replace Hadoop — it typically runs on top of Hadoop, using HDFS for storage and YARN for resource management, while replacing only the slow MapReduce processing engine.

Spark Component	Purpose
Spark Core	Foundation of all components — memory management, fault recovery, task scheduling
Spark SQL	SQL queries on structured data using DataFrame API; reads from Hive, HDFS, JSON, CSV
Spark Streaming	Near-real-time processing via MICRO-BATCHING (collects data every few seconds, not truly instant)
MLlib	Built-in distributed machine learning library — trains ML models across the cluster
GraphX	Processing graph-structured data (social networks, connection analysis)

Feature	Hadoop MapReduce	Spark	Hive
Processing	Disk-based (slow)	In-memory RAM (fast)	Converts queries to MapReduce or Spark
Speed	Baseline (slowest)	Up to 100x faster	Slow (depends on underlying engine)
Streaming	Not supported	Yes — via micro-batching	Not supported
ML Support	Not built-in	Yes — MLlib	Not supported
Query Language	Java programs	Scala, Python, Java, R	HiveQL (SQL-like)
Created By	Doug Cutting (2006)	UC Berkeley AMPLab (2009)	Facebook (2008)

■ **Spark does NOT replace Hadoop — it complements it. Spark uses HDFS (storage) + YARN (resources) and replaces only MapReduce (processing).**

■ **Spark Streaming = MICRO-BATCH (processes every few seconds). Apache Flink = TRUE real-time (processes each event instantly). Common exam trap.**

■ YARN = Yet Another Resource Negotiator — Hadoop's resource manager (added in Hadoop 2.0). Manages CPU and memory allocation across the cluster.

■ PySpark = Python API for Apache Spark. Allows Python developers to write Spark jobs without learning Scala. Kafka + Spark Streaming are often combined: Kafka delivers the data stream, Spark processes it.

TOPIC 10 · BIG DATA PROGRAMMING LANGUAGES — PYTHON, JAVA & SCALA

The languages that power the Big Data ecosystem

◆ Why These Three Languages?

Big Data tools were not built in a vacuum — they were built in specific programming languages, and those languages became dominant in the ecosystem. Hadoop is written in Java, which made Java the original language of Big Data. Spark is written in Scala, making Scala the native language for highest-performance Spark development. Python entered later but became dominant in data analytics and machine learning because of its simplicity and rich library ecosystem. Understanding why each language matters — and what role it plays — is more important for the exam than memorising syntax.

Feature	Python	Java	Scala
Created By	Guido van Rossum	James Gosling (Sun)	Martin Odersky (EPFL Switzerland)
Year	1991	1995	2004
Name Origin	Monty Python comedy	Java coffee (cup logo)	Scalable Language
Runs On	Python interpreter	JVM (Java Virtual Machine)	JVM (Java Virtual Machine)
Type System	Dynamically typed	Statically typed	Statically typed
Execution	Interpreted (slower)	Compiled to bytecode (fast)	Compiled to bytecode (fast)
Learning Curve	Very easy	Moderate	Difficult (steep)
Big Data Role	Analytics, ML, PySpark	Hadoop MapReduce, enterprise	Native Spark applications
Key Framework	PySpark, TensorFlow	Hadoop, Kafka	Apache Spark
Code Style	Very concise	Very verbose	Concise

◆ Why Python Dominates Despite Being Slower

Python is interpreted (slower) and dynamically typed — which can cause runtime errors. Java and Scala are compiled and faster. Yet Python dominates Big Data analytics today. The reason: PySpark. PySpark is the Python API for Apache Spark — it lets Python developers write Spark jobs, and the actual heavy processing is done by Spark's Scala engine under the hood. Python handles the logic; Scala handles the computation. Combined with Python's enormous library ecosystem (Pandas for data manipulation, Scikit-learn for ML, TensorFlow for deep learning), Python became the language of choice for data scientists who do not want to learn Scala.

◆ Python's Key Big Data and AI Libraries

Library	Purpose
NumPy	Fast numerical computing — operations on large arrays and matrices
Pandas	Data manipulation and analysis — works like a programmable Excel in Python
Matplotlib	Data visualisation — creating charts, graphs, and plots
Scikit-learn	Classical machine learning algorithms (classification, regression, clustering)
TensorFlow	Deep learning and neural networks — created by Google
PyTorch	Deep learning — created by Facebook/Meta, popular in research
PySpark	Python API for Apache Spark — write distributed Spark jobs in Python

■ **Spark is written in Scala — but you do NOT need to learn Scala to use Spark. PySpark lets you use Spark in Python. Your classmate saying 'you must learn Scala for Spark' is wrong.**

■ **Scala is NOT a scripting language. It is a full compiled language that runs on the JVM, just like Java.**

■ Java and Scala both run on JVM. Python does NOT run on JVM. JVM = Java Virtual Machine — allows 'write once, run anywhere' across operating systems.

■ Dynamically typed (Python) = you do not declare variable types; Python figures it out at runtime. Statically typed (Java/Scala) = you must declare types explicitly; errors caught at compile time before the program runs.

■ R language = designed for statistical computing. Used by researchers and statisticians. Spark supports R via SparkR.